Software Effort Estimator

Technical Documentation

How the Model Works — From Raw Inputs to Explainable Predictions

METHODOLOGY	Sehra et al. (2019)-inspired weighted RBF-SVR + leakage-safe OOF KernelSHAP
TRAINING DATA	COCOMO-81 (63 projects) · GitHub open-source (91 repos)
VALIDATION	Nested 10-fold outer / 5-fold inner CV · popularity hold-out · NASA-93 external test
HEADLINE MMRE	0.3910 (COCOMO-81) · 0.2055 (GitHub)
RESEARCH CONTEXT	Mitacs Accelerate Internship — Industry partner model deliverable

Keshav · Mitacs Research Demo · 2026

1. What Is Software Effort Estimation?

Software effort estimation answers one question: **how many person-months will it take to build this project?** Getting this right matters enormously. Underestimate and the team burns out trying to meet an impossible deadline. Overestimate and resources sit idle while competitors ship faster.

For decades, companies relied on rules of thumb, gut feel, or simple spreadsheets. The COCOMO-81 model (Boehm, 1981) was a landmark — the first systematic, empirically grounded approach. It said: *if you tell me the size of the codebase and 15 project attributes, I can give you a reliable effort estimate.*

This research prototype builds on that foundation in three ways:

- **Training-derived weighting** — a data-derived FAHP-inspired proxy assigns feature weights inside each training fold. It is not an independent panel of expert judgements.
- **Non-linear learning** — Support Vector Regression (SVR) with a weighted kernel learns patterns from historical data and is evaluated separately from the original multiplicative COCOMO formula.
- **Explainability** — KernelSHAP attributes each model output to its inputs, in person-months, relative to a stated background dataset. These are model attributions, not causal effects.

> This is an adaptation, not a reproduction, of Sehra et al. (2019): their study used FAHP-weighted LSSVM, whereas this prototype uses a training-derived weighting proxy with sklearn SVR and adds an out-of-fold KernelSHAP analysis.

1.1 The Core Metric: MMRE

Throughout this document you will see **MMRE** (Mean Magnitude of Relative Error). It is the standard accuracy measure for effort estimation:

$$\text{MMRE} = (1/n) \times \Sigma \, |\text{actual_effort} - \text{predicted_effort}| \, / \, \text{actual_effort}$$

An MMRE of 0.30 means predictions are, on average, 30% away from the true value. Lower is better. PRED(25) is the complementary metric: the fraction of predictions within 25% of the true value. Higher is better.

MMRE	What it means	Context
< 0.10	Excellent — within 10% on average	Rarely achieved on real projects
0.10 – 0.25	Lower relative error in this sample	GitHub nested-CV result: 0.2055
0.25 – 0.40	Moderate relative error in this sample	COCOMO nested-CV result: 0.3910
> 0.50	Poor — coin-flip territory	Typical spreadsheet estimation

2. The COCOMO-81 Foundation

COCOMO stands for COnstructive COst MOdel. Barry Boehm published it in 1981 after analysing 63 software projects at TRW Aerospace. The fundamental equation is:

$$\text{Effort (PM)} = a \times \text{KLOC}^b \times \text{EAF}$$

Where:

- **KLOC** = thousands of lines of code (project size)
- **a, b** = constants that depend on development mode (Organic / Semi-detached / Embedded)
- **EAF** = Effort Adjustment Factor — a product of 15 cost driver multipliers

2.1 Development Modes

Mode	Description	a	b	Typical project
Organic	Small team, familiar domain, flexible requirements	2.4	1.05	Internal tools, small web apps
Semi-detached	Mixed experience, moderate constraints	3.0	1.12	Mid-size commercial software
Embedded	Hard real-time, safety-critical, tight hardware constraints	3.6	1.20	Avionics, medical devices

2.2 The 15 Cost Drivers (EAF)

Each cost driver is rated on a 5-point scale: Very Low, Low, Nominal, High, Very High. Each rating maps to a multiplier. The product of all 15 multipliers is the EAF. EAF < 1.0 means the project conditions are **favourable** (reducing effort); EAF > 1.0 means they are **challenging** (increasing effort).

Category	Driver	What it measures	VL → VH range
Product	RELY	Consequence of failure (inconvenience → safety-critical)	0.75 → 1.40
Product	DATA	Database and data complexity	0.94 → 1.16
Product	CPLX	Algorithmic and architectural complexity	0.70 → 1.30
Computer	TIME	CPU execution-time constraint	1.00 → 1.30
Computer	STOR	Memory constraint	1.00 → 1.21
Computer	VIRT	Platform/VM volatility	0.87 → 1.30
Computer	TURN	Development turnaround time	0.87 → 1.15
Personnel	ACAP	Analyst/architect capability	1.46 → 0.71
Personnel	AEXP	Application domain experience	1.29 → 0.82

Category	Driver	What it measures	VL → VH range
Personnel	PCAP	Programmer capability	1.42 → 0.70
Personnel	VEXP	Virtual machine experience	1.21 → 0.87
Personnel	LEXP	Programming language experience	1.14 → 0.95
Project	MODP	Modern programming practices	1.24 → 0.82
Project	TOOL	Software tools quality	1.24 → 0.83
Project	SCED	Schedule constraint	1.23 → 1.10

Example: A 50 KLOC Organic project with all Nominal ratings gives:
Effort = 2.4 × 50^1.05 × 1.0 = 2.4 × 60.8 = **~146 person-months**
If the team has Very High programmer capability (PCAP multiplier = 0.70) and High complexity (CPLX = 1.15):
EAF = 0.70 × 1.15 = 0.805 → Effort = 146 × 0.805 = **~118 person-months**

3. Fuzzy Analytic Hierarchy Process (FAHP)

COCOMO uses calibrated effort multipliers whose ranges differ by cost driver; it does not provide a separate, learned ranking of feature importance. Sehra et al. (2019) addressed feature ranking with FAHP before integrating the resulting weights into an LSSVM. The present sklearn-SVR implementation uses a **training-derived FAHP-inspired proxy** and must not be described as a faithful reproduction of that method.

3.1 What Is the Analytic Hierarchy Process?

AHP (Saaty, 1980) is a decision-making framework that assigns weights to criteria by asking: *how much more important is criterion A than criterion B?* You build a matrix of pairwise comparisons and extract weights mathematically.

The problem with standard AHP is that human judgements are imprecise. Saying "CPLX is 3× more important than LEXP" is hard to justify exactly. **Fuzzy AHP** replaces exact numbers with triangular fuzzy numbers (TFNs) that capture this uncertainty.

That is the motivation for expert-elicited FAHP. No independent expert panel was available for this prototype, so its current weights are supervised, data-derived proxies. A paper should keep that distinction explicit.

3.2 Triangular Fuzzy Numbers

A TFN is written as (l, m, u): a lower bound, a most likely value, and an upper bound. Instead of saying "A is exactly 3× more important than B", we say "A is between 1× and 5× more important, most likely 3×" — written as (1, 3, 5).

$$TFN = (l, m, u) \text{ where } l \le m \le u$$

The TFN scale used in this model (Buckley 1985 / Chang 1996):

Linguistic judgement	TFN
Equally important	(1, 1, 1)
Weakly more important	(1, 2, 3)
Moderately more important	(1, 3, 5)
Strongly more important	(3, 5, 7)
Very strongly more important	(5, 7, 9)
Absolutely more important	(7, 9, 9)

3.3 Training-Derived FAHP-Inspired Proxy Used Here

The implemented COCOMO weighting procedure operates only on the relevant training rows and has three stages:

1. **Step 1 — Project comparisons:** Build separate fuzzy pairwise matrices across training projects from their effort and LOC ratios; outer-test outcomes never enter these matrices.
2. **Step 2 — Project weights:** Apply Chang's synthetic-extent and degree-of-possibility calculations to obtain effort-based and LOC-based project weights, then average and normalise them.

$$S_i = (\Sigma_j l_{ij} / \Sigma_i \Sigma_j u_{ij} , \Sigma_j m_{ij} / \Sigma_i \Sigma_j m_{ij} , \Sigma_j u_{ij} / \Sigma_i \Sigma_j l_{ij})$$

3. **Step 3 — Feature proxy weights:** Take the absolute Spearman association between each feature and the combined project weights, clip degenerate values, and normalise across features.

$$\theta_i = \text{normalise}(|\text{Spearman}(\text{feature}_i, \text{combined training-project weight})|)$$

Scope: These weights are useful as a supervised ranking and kernel-scaling mechanism, but they are not independent expert priors. A faithful replication-and-extension study should obtain the original expert judgements or elicit a new documented panel.

3.4 How the Proxy Enters the Kernel

The normalised proxy vector theta is applied as an element-wise square-root transformation before the RBF kernel. This makes theta the coefficient of each feature's squared-distance contribution. Predictive benefit is assessed separately with a matched, independently tuned weighted-versus-unweighted

nested-validation comparison.

4. Weighted-Kernel Support Vector Regression (SVR)

Once the training-derived proxy produces feature weights, an SVR learns non-linear patterns from the 63 COCOMO-81 projects. This is sklearn epsilon-SVR, not the LSSVM solver used by Sehra et al. (2019).

4.1 What Is SVR?

Support Vector Regression learns a function that predicts continuous outputs (like person-months). Unlike linear regression, it uses a **kernel function** to measure similarity between projects in a high-dimensional feature space, allowing it to capture non-linear relationships.

The key idea: instead of minimising the sum of squared errors (like OLS regression), SVR tries to find a function where most predictions fall within an ε-tube around the true values. Points outside the tube are penalised by a cost parameter C.

$$\text{SVR finds } f(x) \text{ such that } |y_i - f(x_i)| \leq \varepsilon \text{ for most training points}$$

4.2 The RBF Kernel

This model uses the Radial Basis Function (RBF) kernel, which measures how "close" two projects are in feature space:

$$K(x_i, x_j) = \exp(-\gamma \times ||x_i - x_j||^2)$$

Two projects that are very similar (small distance) have $K \approx 1$. Two projects that are very different have $K \approx 0$. The model learns that similar projects should have similar effort estimates. γ controls how quickly similarity falls off with distance — a hyperparameter tuned by cross-validation.

4.3 Training-Derived Weighting in the RBF Kernel

Instead of measuring distance equally across all features, the implementation rescales the feature space with the training-derived proxy weights:

$$K_{\theta}(x_i, x_j) = \exp(-\text{gamma} \times ||\sqrt{\theta} \cdot (x_i - x_j)||^2)$$

Where theta is the vector of normalised proxy weights and sqrt(theta) is applied element-wise. This is equivalent to:

$$x_{\text{weighted}} = x \times \sqrt{\theta}$$

A larger theta therefore gives that feature a larger coefficient in squared RBF distance. This changes the model's geometry; it does not establish that the feature is a causal effort driver.

4.4 The Full Training Pipeline

Step	What happens	Why
1. Log transform	x → log(x), y → log(1 + y)	Effort is right-skewed; log space makes it more symmetric
2. Scale	MinMaxScaler to [0, 1]	SVR is sensitive to feature scale differences
3. Proxy weight	x_weighted = x_scaled × sqrt(theta)	Rescales each feature's contribution to RBF distance
4. SVR fit	SVR(RBF, C, γ, ε).fit(X_weighted, y_scaled)	Learn non-linear effort surface
5. Predict	Predict → inverse-scale → expm1	Convert back from log-space to person-months

4.5 Hyperparameter Tuning

Three SVR hyperparameters are tuned by exhaustive grid search, using five-fold inner-CV MMRE as the objective inside each outer-training partition:

- **C** (cost): penalty for predictions outside the ε-tube. Higher C = tighter fit, risk of overfit.
- **γ** (gamma): how quickly the RBF kernel falls off. Higher γ = more local decisions.
- **ε** (epsilon): the width of the no-penalty tube. Larger ε = more tolerance.

Why nested validation? A single resampling loop cannot both choose hyperparameters and provide an untouched performance estimate. Ten outer folds are used only for evaluation; within each outer-training partition, five inner folds select C, gamma and epsilon. Scaling and training-derived proxy weights are also fitted only on the relevant training rows. Deployment parameters are selected separately after evaluation.

5. Out-of-Fold KernelSHAP Analysis

SHAP (SHapley Additive exPlanations; Lundberg and Lee, 2017) decomposes a **model output** relative to an expected output defined by a background dataset. Because the complete prediction function returns person-months, these attributions are also expressed in person-months. They describe model behaviour, not real-world causality.

model prediction = expected model output over background + sum of SHAP attributions

5.1 Leakage-Safe Repeated Protocol

The research analysis uses **10 repeats of 5-fold outer validation**, with 3-fold inner selection. In every outer fold, preprocessing, the training-derived weighting proxy, and hyperparameters are fitted without

the outer-test projects. KernelSHAP then explains only those outer-test predictions, so every project is explained once per repeat.

- **Explainer:** model-agnostic KernelSHAP over the complete raw-input prediction pipeline
- **Background:** all outer-training projects; outer-training partition only; outcomes are not used to select background rows
- **Sampling:** nsamples=256, l1_reg=0; all 17 features enter the weighted least-squares solution
- **Additivity check:** maximum stored error = 0.000000 person-months

Primary comparison: the training-derived FAHP-inspired proxy is compared with normalised mean absolute out-of-fold SHAP reliance from an *unweighted* SVR. This is the cleaner comparison because the proxy weights were not injected into that model. Agreement with weighted-model SHAP is reported separately and is descriptive: some of it may be induced by the weighted kernel itself.

5.2 Agreement Results

Comparison	Spearman rho	Kendall tau-b	JS similarity	Top-3 Jaccard / Kuncheva
Primary pooled: proxy vs unweighted OOF SHAP	0.549020	0.352941	0.758709	0.200000 / 0.190476 (1/3 shared)
Primary repeat median [min, max]	0.471814 [0.404412, 0.651961]	0.330882 [0.250000, 0.470588]	0.755400 [0.734730, 0.771563]	0.350000 / 0.392857
Weighted pooled (descriptive; partly induced)	0.710784	0.500000	0.772099	0.500000 / 0.595238 (2/3 shared)
Weighted repeat median [min, max]	0.658088 [0.406863, 0.732843]	0.470588 [0.308824, 0.558824]	0.764367 [0.739879, 0.804626]	0.500000 / 0.595238

The primary pooled rank association is moderate, while only one of the three highest-ranked features is shared. The larger weighted-model association must not be read as independent confirmation because the proxy weights alter that model's geometry.

A 2,000-resample project-level percentile procedure gives a conditional 95% interval for primary Spearman rho of [0.401961, 0.607843] (resampling median 0.514706). Projects are resampled after averaging their repeated OOF explanations. It does not refit models, regenerate splits, retune hyperparameters, or rerun the explainer, so it is not a full uncertainty interval.

5.3 Repeat Stability

Primary unweighted evidence	Pairs	Statistic	Median [min, max]
Joint repeat ranking	45	Spearman rho	0.921569 [0.811275, 0.968137]
Joint repeat ranking	45	Kendall tau-b	0.794118 [0.617647, 0.882353]
Joint repeat ranking	45	Top-3 Jaccard	0.500000 [0.200000, 1.000000]
Joint local attribution	2835	Cosine similarity	0.957340 [0.137241, 0.998259]
Joint local attribution	2835	Sign agreement	0.823529 [0.470588, 1.000000]
Joint local attribution	2835	Top-3 Jaccard	0.500000 [0.000000, 1.000000]

The split, fitted model, outer-training background, and KernelSHAP seed vary together, so these are joint resampling stability measurements rather than isolated model or explainer sensitivity. For the descriptive weighted model, joint repeat stability has median Spearman rho 0.894608 [0.637255, 0.963235], and local cosine similarity has median 0.963318 [0.152390, 0.998180]. The low minima show why a single explanation should not be presented as universally stable.

A separate fixed-model numerical audit holds the model, background, and 12 rows fixed. At nsamples=256 versus the 1024-sample reference, global Spearman rho is 1.000000, Jensen-Shannon similarity is 0.989210, the top three are identical, and median local cosine similarity is 0.999547. Seed-to-seed median global rho is 0.997549. With l1_reg=0, every audited local explanation retains all 17 features.

In a 10,000-permutation feature-label benchmark, primary Spearman agreement has empirical p=0.025797 and Jensen-Shannon similarity p=0.003700, while top-3 Jaccard overlap is not unusual (p=0.462754). This is a narrow descriptive relabelling benchmark, not a population hypothesis test.

5.4 Highest Unweighted OOF SHAP Reliance

Feature	Proxy weight (rank)	Unweighted OOF SHAP (rank)	Weighted OOF SHAP (rank)
LOC (KLOC)	0.226659 (#1)	0.329450 (#1)	0.373351 (#1)
RELY	0.044488 (#7)	0.072291 (#2)	0.059705 (#5)
TIME	0.041998 (#8)	0.069630 (#3)	0.068650 (#3)
Dev Mode	0.114108 (#3)	0.069218 (#4)	0.091528 (#2)
PCAP	0.054818 (#6)	0.067262 (#5)	0.067857 (#4)

OOF SHAP reliance is normalised mean absolute attribution. It is not a causal effect, and correlated COCOMO drivers may redistribute attribution. LOC ranks first in both the proxy and primary SHAP result; RELY and TIME rank second and third by unweighted OOF SHAP even though their proxy ranks are lower.

5.5 Relation to Prior Explainability Work

Adding SHAP to software-effort estimation is not by itself novel. Najm, Zakrani, and Marzak (2023) reported model-agnostic explanations for SVR-RBF effort models, and Jayadi, Prasetya, and Widiyaningtyas (2025) applied SHAP to a simpler effort model. This pilot instead evaluates a specific FAHP-proxy/SHAP relationship with repeated, leakage-safe out-of-fold explanations and an unweighted primary comparator. It does not establish a first-in-field claim.

Interactive-app distinction: the on-screen single-project explanation uses a fixed, outcome-conditioned 50-project background (projects with effort at or below 600 PM), nsamples=256, and l1_reg=0. Its base value is the expected model output over those selected rows, not a representative population mean. The research results above instead use all outer-training rows as each fold's background.

6. Post-hoc GenAI Sensitivity Scenario

COCOMO-81 was published in 1981. Its TOOL cost driver captures the quality of development tools; the multiplier at Very High is 0.83. Modern AI coding tools are not represented in the historical training data.

6.1 External Sources Used for Illustrative Bounds

Source	Reported setting	Reported association	Use here
Peng et al. (2023) — GitHub	Controlled coding task	Faster task completion with Copilot; full-project effort was not measured	Illustrative bound only
McKinsey Global Institute (2023)	Industry report	A range of potential developer-productivity effects	Illustrative bound only
Kalliamvakou (2022) — GitHub blog	Observed Copilot cohort	Faster pull-request merging and self-reported experience	Illustrative bound only
COCOMO-81 TOOL driver	Historical projects	Very High TOOL multiplier is 0.83	Already represented in model input

Research question: Sehra et al. (2019) did not evaluate modern AI-tool signals. The multipliers below are external, post-hoc scenarios rather than learned effects. Estimating any AI-associated change from company projects with recorded effort would be a future empirical contribution; this prototype does not claim that result yet.

6.2 How the Adjustment Works

The GenAI adjustment is a **post-hoc multiplier** applied to the SVR prediction. It is not baked into the model — it is applied on top, transparently:

$$AI_effort = baseline_effort \times AI_multiplier$$

AI level	Multiplier (central)	Range	Source
None — no AI tools	1.00	—	—
Some — Copilot / ChatGPT	0.78	0.75–0.82	Peng et al. 2023, McKinsey 2023
Heavy — Cursor / AI-first team	0.62	0.55–0.70	McKinsey 2023, Kalliamvakou 2022

The range is shown as a scenario range in the UI, reflecting variation across the cited studies. Under the illustrative "Heavy AI" central multiplier, a 200 PM model output becomes 200 × 0.62 = **124 PM**, a modelled difference of 76 PM (38%). Applying the scenario bounds gives 110–140 PM. These are arithmetic sensitivity results, not a causal forecast or a confidence interval.

Important caveat: These multipliers come from task-level studies and enterprise surveys, not from effort estimation research. Applying them to full-project effort estimates is an unvalidated extrapolation. The actual effect depends on the codebase, team, adoption, and task mix and must be measured independently.

7. The GitHub-Trained Model

The COCOMO-81 model was trained on historical defense software. As an exploratory modern-data exercise, 91 public GitHub repositories were used to train an RBF-SVR with supervised correlation-based feature weighting. This is not the same FAHP-LSSVM method as Sehra et al. (2019), and it does not establish industry generalisation.

7.1 Features

Feature	What it measures	Possible association (not causal)
Contributors	Unique people who committed	Team activity and the effort proxy share commit-derived inputs
PR Review Time	Avg days from PR open to merge	May reflect workflow latency or repository practice
PR Merge Rate	Fraction of PRs merged	May reflect review and contribution policy
Test File Ratio	Fraction of files that are tests	Repository-structure signal; not measured test quality
Language Count	Number of programming languages	Repository heterogeneity signal

Feature	What it measures	Possible association (not causal)
Commit Frequency	Commits per week	Development-activity signal
Contributors × Freq	Interaction term	Combined team-activity scale
AI Tool Adoption	Number of detectable AI configs	Configuration presence; adoption intensity is unknown

7.2 The Effort Proxy

GitHub doesn't have Jira tickets with actual hours. Instead, effort is approximated as:

$$\text{author_months} = \text{unique (author, date) pairs across all commits} / 20$$

A contributor who commits 10 times in one day still counts as 1 author-day. Dividing by 20 converts working days to months (approx. 20 working days/month). This is an imperfect proxy — it measures developer presence, not hours of productive work — and it has not been validated against recorded person-hours.

Sampling ceiling: collection is capped at 1,200 commits per repository, approximately 60 author-months under this proxy. Predictions above that cap are extrapolations and have not been validated.

7.3 Transparency: Feature/Target Overlap

Since contributor_count, the interaction term, and author_months all derive from the same commit-activity log, a reviewer should ask whether the low MMRE reflects genuine predictive signal or shared measurement source. We checked all three key correlations:

Feature	Spearman r with author_months	Note
contributor_count	+0.705	High — both derived from who committed
contributors_x_freq	+0.650	High — interaction amplifies the overlap
commit_frequency	+0.395	Moderate

This is **construct overlap, not train/test leakage.** Strict leakage means test-set information bleeding into training, inflating CV scores — the CV here is clean. What exists is that author_months deduplicates to unique person-days and divides by 20, so it is strongly related to — but not a deterministic function of — contributor count. A 50-person team each committing once total has very different author_months than 50 people each active for months. The r = 0.70 says they are strongly correlated, not identical.

The low MMRE therefore reflects a predictive association that is entangled with a shared measurement source; it does not establish that team size causes effort. Jira-tracked hours or another independent ground truth would permit a cleaner accuracy evaluation.

An earlier version of this section reported Spearman(commit_frequency, author_months) = 0.04 and concluded "no meaningful leakage exists." That figure was computed with Pearson correlation, not Spearman. The corrected Spearman values are shown in the table above. The conclusion changes: construct overlap exists in the dominant feature (r = 0.70), and the MMRE should be interpreted accordingly — as a strong proof-of-concept, not as a clean accuracy benchmark.

7.4 Popularity Hold-out (Not Temporal Validation)

The current CSV has no trustworthy project-date field, so a chronological test cannot be claimed. The available sensitivity check sorts repositories by star count. Stars measure popularity, not time:

- Train on the higher-star partition
- Test on the lower-star partition
- Select hyperparameters with inner five-fold CV on the training partition only
- Re-fit scalers and supervised correlation weights entirely on training data

Validation method	MMRE	PRED(25)	Notes
Nested 10x5 CV	0.2055	70.3%	Headline GitHub estimate
Popularity hold-out	0.1992	64.3%	High-star train to low-star test (n=28)
COCOMO-81 nested CV	0.3910	46.0%	Separate historical benchmark

This result is a popularity sensitivity check only. It must not be described as temporal generalisation. Future collection records repository creation and sampled commit dates so a genuinely chronological evaluation can be performed.

8. Validation Results

8.1 Ablation Study — Does Training-Derived Weighting Help?

A fair ablation must give the unweighted baseline its OWN hyperparameter search. Reusing the C/gamma/epsilon tuned for the weighted feature space disadvantages the baseline and inflates the apparent benefit. Below, each variant is independently tuned over the same grid, with leakage-free nested evaluation:

Model variant	Nested-CV MMRE	Weighting comparison
Plain SVR (unweighted, independently tuned)	0.4312	baseline
Proxy-weighted SVR	0.3910	9.3% lower MMRE in this comparison

In matched nested validation, training-derived proxy weighting changes COCOMO-81 MMRE by **9.3%** (0.4312 to 0.3910). This is a sample-specific comparison, not proof that weighting improves every future

dataset.

Across the 10 repeated 5-outer/3-inner runs used for the explainability study, the weighted model has lower MMRE in 9/10 repeats. Median weighted MMRE is 0.425209 [0.349366, 0.575162], versus 0.502545 [0.402853, 0.706923] unweighted. The median paired difference (unweighted minus weighted) is +0.080563 [-0.015046, +0.135790]. Median MdMRE is 0.307316 weighted versus 0.361205 unweighted, with paired difference +0.037036. Median MAE is 208.131571 versus 245.059008 person-months, with paired difference +19.813137 PM. Repeated partitions overlap and are not independent inferential units, so this is descriptive; no uncorrected significance test is claimed.

GitHub comparison: the independently tuned unweighted model scores 0.2149, while the weighted model scores 0.2055, a 4.4% lower MMRE in these nested folds. The result is reported as a comparison because it does not establish a universal causal improvement from feature weighting.

8.2 Cross-Dataset Generalisation (NASA-93)

To test whether the COCOMO-81 model generalises beyond its training distribution, we evaluate it on NASA-93 — 93 entirely different NASA software projects collected 12 years later. No NASA-93 data was used in training or tuning.

Evaluation	Dataset	n	MMRE	PRED(25)
Nested 10x5 CV	COCOMO-81	63	0.3910	46.0%
External test	NASA-93	93	0.5175	46.2%

MMRE is 0.3910 in nested COCOMO-81 validation and 0.5175 on NASA-93. The external result measures transfer to a separate dataset; it does not guarantee performance for modern commercial projects.

8.3 The 80% Prediction Interval

Beyond point estimates, the tool shows an 80% prediction interval — a range where approximately 4 out of 5 real projects should fall. This is computed empirically from pure nested outer-fold residuals (no parametric assumption):

$$\text{error} = (\text{prediction} - \text{actual}) / \text{actual}$$
$$\text{PI} = [\text{prediction} / (1 + q90), \text{prediction} / (1 + q10)]$$

For COCOMO-81, q10=-0.443 and q90=+0.680. If the point prediction is 100 PM, the transformed 80% interval is [59.5, 179.5] PM. Roughly one in five comparable projects may fall outside this empirical range.

8.4 Per Development Mode Breakdown

Not all projects are equally predictable. Splitting the COCOMO-81 results by development mode reveals an interesting pattern:

Mode	n	Nested-fold MMRE	PRED(25)	Notes
Organic	24	0.3317	54.2%	Small familiar projects
Semi-detached	11	0.3757	27.3%	Mixed experience
Embedded	28	0.4479	46.4%	Hard real-time or constrained

These subsets are small, especially semi-detached (n=11). Their metrics are descriptive breakdowns of the outer-fold predictions, not independent validation studies.

> **Takeaway:** Use per-mode numbers as context, not a strict trust ranking. Sampling variation can be substantial at these group sizes.

9. End-to-End Worked Example

Let's walk through a complete prediction for a realistic project: a **FinTech payment processing API**, 40 KLOC, Semi-detached mode, with a moderately experienced team, some schedule pressure, and Copilot adopted.

Step 1 — Gather Inputs

Input	Value	Rationale
LOC	40 KLOC	Mid-size API service
RELY	High (1.15)	Financial transactions — SLA breach = money loss
DATA	Nominal (1.00)	Standard database complexity
CPLX	High (1.15)	Real-time settlement logic
TIME	Nominal (1.00)	Standard latency requirements
STOR	Nominal (1.00)	Cloud deployment, no memory pressure
VIRT	Low (0.87)	Stable cloud infrastructure
TURN	Low (0.87)	Fast CI/CD pipeline
ACAP	High (0.86)	Strong architects
AEXP	Nominal (1.00)	Some FinTech experience
PCAP	High (0.86)	Solid developer team
VEXP	High (0.90)	AWS-experienced team

Input	Value	Rationale
LEXP	Nominal (1.00)	TypeScript / Node.js — familiar
MODP	High (0.91)	Good Agile practices
TOOL	High (0.91)	Copilot + good tooling
SCED	High (1.04)	Fixed product launch date
Dev Mode	Semi-detached	Mixed team, moderate constraints

Step 2 — Compute the Raw COCOMO Estimate

EAF = 1.15 × 1.00 × 1.15 × 1.00 × 1.00 × 0.87 × 0.87 × 0.86 × 1.00 × 0.86 × 0.90 × 1.00 × 0.91 × 0.91 × 1.04

EAF ≈ 0.574 (favourable team/tools conditions reducing the baseline)

Baseline (Semi-det) = 3.0 × 40^1.12 = 3.0 × 62.3 = 186.9 PM COCOMO Effort = 186.9 × 0.574 ≈ 107 PM

Step 3 — SVR Prediction (What the Model Actually Returns)

The SVR doesn't use the COCOMO formula directly. It:

- Converts the raw inputs (LOC, 15 multipliers, dev mode ordinal) to a feature vector
- Log-transforms, MinMax-scales, and applies the training-derived proxy weights
- Passes it through the trained RBF kernel SVR
- Inverse-scales and expm1 transforms back to person-months

For these exact inputs the model returns **111 PM** — nearly identical to the COCOMO formula's 107 PM. This is expected: with a well-calibrated EAF (0.574) on a Semi-detached project squarely within the training distribution, the SVR and the COCOMO multiplicative formula converge. The SVR adds value on projects where non-linear interactions between drivers matter more — here, the drivers are moderate and the two methods agree.

Step 4 — Illustrative Local SHAP Snapshot (seed=42, fixed background n=50)

Background: the 50 training projects with effort ≤ 600 PM, selected for a responsive display explanation. Because that selection uses the outcome, this background is illustrative rather than representative. The resulting base value in the frozen run is **115.8 PM**, and the SHAP values are model-output attributions in PM units. This frozen example had 0.00 PM stored additivity error; it is not the repeated OOF experiment in Section 5.

Feature	SHAP value	Reading
LOC (40 KLOC)	+69.7 PM	Raises this model output relative to the fixed background
RELY (High)	+14.0 PM	Raises this model output relative to the fixed background
DevMode (Semi)	+7.6 PM	Raises this model output relative to the fixed background
TOOL (High)	−23.5 PM	Lowers this model output relative to the fixed background
TIME (Nominal)	−21.8 PM	Lowers this model output relative to the fixed background
TURN (Low)	−15.1 PM	Lowers this model output relative to the fixed background
PCAP (High)	−12.4 PM	Lowers this model output relative to the fixed background
VEXP (High)	−11.3 PM	Lowers this model output relative to the fixed background
ACAP (High)	−7.7 PM	Lowers this model output relative to the fixed background
MODP (High)	−4.6 PM	Lowers this model output relative to the fixed background
All others	0.0 PM	Negligible attribution for this input/background/model combination
Baseline	115.8 PM	Expected model output over the selected 50-project display background (seed=42)
Total	**110.8 PM**	Frozen illustrative run: 115.8 − 5.0 = 110.8; rounded additivity residual 0.00%

Step 5 — Indicative Schedule and Staffing Ratio

Calendar months = 2.5 × 111^0.38 = 2.5 × 5.98 ≈ 15 months
Implied average staffing = 111 / 15 ≈ 7 engineers

Step 6 — Post-hoc GenAI Sensitivity Scenario

Scenario effort = 111 × 0.78 = 87 PM (assumed 22% multiplier)
AI calendar = 2.5 × 87^0.38 = 2.5 × 5.46 ≈ 14 months | AI team = 6 engineers
80% PI: 111 / (1 + +0.680) = 66 PM to 111 / (1 + -0.443) = 199 PM

Summary for the FinTech API project (illustrative model output):
SVR prediction: 110.8 PM · COCOMO: 107 PM · estimates are close for this illustrative input
Without AI: 111 PM, 15 months, 7 engineers
With the assumed Copilot adjustment: 87 PM, 14 months, 6 engineers (modelled difference ~24 PM = 22%; not a causal forecast)
80% empirical interval: 66–199 PM (wide — reflects genuine estimation uncertainty)
SHAP baseline: 115.8 PM (expected model output over the selected 50-project display background, seed=42)
Largest positive local attribution: LOC (+69.7 PM); largest negative local attributions: TOOL (−23.5 PM), TIME (−21.8 PM)

10. Limitations and Scope of Validity

No model is perfect. Being transparent about limitations is a sign of rigorous research, not weakness. Use these estimates as structured guidance, not budget commitments.

10.1 COCOMO-81 Model

- **Training data age:** 63 projects from the 1980s, mostly defense/aerospace. The model has never seen React, microservices, CI/CD, or cloud-native development.
- **LOC as a proxy:** Modern teams often work with story points, not LOC. The model accepts LOC as a proxy for scope — use sprint velocity × total sprints as an approximation.
- **Not a reproduction:** Sehra et al. (2019) used FAHP-weighted LSSVM; this implementation uses sklearn epsilon-SVR with a mathematically analogous weighted RBF transformation.
- **Weight derivation:** Independent expert pairwise comparisons are replaced with a supervised, data-derived FAHP-inspired proxy. A faithful replication-and-extension study requires the original judgements or a documented new expert panel.
- **Proxy sparsity and consistency:** Full-data Chang extent weights are nonzero for 25/63 effort projects, 23/63 LOC projects, and 30/63 combined weights. The effort/LOC pairwise consistency ratios are 0.105267/0.054688; proxy sensitivity and genuine expert elicitation remain necessary for confirmation.
- **SHAP scope:** Attributions are relative to a chosen training background and are not causal evidence. Correlated COCOMO drivers may redistribute attribution; dependence-aware sensitivity analysis remains future work.
- **Size-response compression.** The model's effort response to project size is slightly sublinear — the effective in-range exponent is ~0.88, versus COCOMO-81's 1.05 — a consequence of RBF-kernel smoothing pulling predictions toward the training distribution. Within the training range this is minor, but the compression intensifies with distance from the data: beyond the largest training projects it produces systematic under-prediction (the basis for the extrapolation warning at 73 KLOC in Organic mode). Large-project estimates should therefore be read as lower bounds, and inputs beyond the per-mode training range treated as unreliable rather than merely uncertain.

10.2 GitHub Model

- **Effort proxy quality:** Author-months from commit logs ≠ actual billed hours. Open-source volunteers often commit in bursts; paid developers have more consistent patterns.
- **Sampling cap:** At most 1,200 commits are sampled per repository (approximately 60 author-months). Predictions above that range are extrapolated and not validated.
- **Open-source bias:** GitHub repos are maintained by volunteers with different motivations than commercial development teams. Coordination overhead and rework patterns may differ.
- **Feature availability:** AI tool adoption is inferred from config files — not all teams configure their tools in discoverable ways, leading to underestimation of adoption.

10.3 GenAI Multipliers

- **Post-hoc adjustment:** Multipliers are applied to total effort. AI tools primarily help with coding-specific tasks (code completion, test generation); planning, stakeholder management, and architecture work may see less benefit.
- **Study variation:** The illustrative range is wide and mixes evidence types. Treat it as sensitivity analysis, not a planning estimate; validate any effect on independent completed-project data.
- **COCOMO TOOL interaction:** The TOOL driver already accounts for some tooling quality. Using both TOOL at Very High and a Heavy AI multiplier may double-count some benefit.

10.4 What the Model Cannot Do

- Predict individual sprint velocity or week-by-week burn rate
- Account for organisational dysfunction, communication failures, or political delays
- Estimate effort for research-phase work where requirements are undefined
- Replace expert estimation — it is a structured input to a human decision, not a replacement for one

11. The Mitacs Vision — Why Company Data Matters

Every model in this tool is limited by the data it was trained on. COCOMO-81 learned from 1980s defense software. The GitHub model learned from open-source volunteers. **Neither knows your team.**

A model trained on your company's completed projects — with actual Jira-tracked hours as the ground truth — would:

- Learn your team's specific velocity, not a 1981 average
- Identify which COCOMO drivers actually predict overruns *for you*
- Discover your chronically underestimated project types
- Estimate AI-tool associations using independently recorded adoption and completed-project effort

11.1 What Can Be Claimed Today

Model	Training data	Validation MMRE	PRED(25)
COCOMO-81 benchmark	63 projects	0.3910 nested CV	46.0%
GitHub proof of concept	91 repos	0.2055 nested CV	70.3%
Your company data	Not yet provided	Not measured	Not measured

Company data could be more representative of the intended use case, but no numerical improvement is promised. Its MMRE and PRED(25) must be measured using held-out completed projects after the target definition and data quality are validated.

11.2 What You Would Provide

- **From Jira:** Story points, sprint velocity, cycle time, bug reopen rate, team size, planned vs actual delivery
- **From GitHub:** PRs merged, review time, test coverage, language count, AI tool adoption, commit frequency
- **Scale:** A small upload can be checked for schema and feasibility, but reliable model validation generally needs substantially more completed projects; uncertainty must be reported for the actual sample.

11.3 The 4-Month Deliverable

Phase	Timeline	Deliverable
Data collection & feature engineering	Month 1–2	Clean dataset with effort ground truth + feature set
Model training & validation	Month 3	Weighted-SVR adaptation, repeated OOF KernelSHAP, nested validation and chronological hold-out when dates permit
Deployment & handoff	Month 4	Live interactive dashboard on your internal infrastructure, documentation, maintenance guide — no sensitive data leaves your environment

11.4 Cost Structure

Contributor	Amount	What it covers
Industry partner	$7,500	4 months of focused research and development
Mitacs (federal funding)	$7,500	Matched 1:1 — equal federal and industry contribution
Total research value	$15,000	Custom AI estimation tool delivered in 4 months

12. References

Reference	Relevance
Boehm, B.W. (1981). Software Engineering Economics. Prentice Hall.	COCOMO-81 model, cost drivers, EAF
Saaty, T.L. (1980). The Analytic Hierarchy Process. McGraw-Hill.	Foundation of AHP
Buckley, J.J. (1985). Fuzzy hierarchical analysis. Fuzzy Sets and Systems, 17(3), 233–247.	Fuzzy TFN scale for pairwise comparisons
Chang, D.Y. (1996). Applications of the extent analysis method on fuzzy AHP. European Journal of Operational Research, 95(3), 649–655.	Extent-analysis component used in the data-derived proxy
Sehra, S.K., Brar, Y.S., Kaur, N., & Sehra, S.S. (2019). Software effort estimation using FAHP and weighted kernel LSSVM machine. Soft Computing, 23(21), 10881–10900. https://doi.org/10.1007/s00500-018-3639-2	Foundational FAHP-weighted LSSVM study; this project is an adaptation, not a reproduction
Peng, S. et al. (2023). The impact of AI coding assistants on developer productivity: A controlled experiment. GitHub Research.	55% faster task completion in the controlled experiment
McKinsey Global Institute (2023). The economic potential of generative AI.	20–45% developer productivity gain
Kalliamvakou, E. (2022). Research: Quantifying GitHub Copilot's impact on developer productivity. GitHub Blog.	Copilot PR merge speed study
Lundberg, S.M., Lee, S.I. (2017). A unified approach to interpreting model predictions. NeurIPS 30.	SHAP explainability framework
Najm, A., Zakrani, A., & Marzak, A. (2023). Efficient Shapely Explanation of Support Vector Regression for Agile and Non-agile Software Effort Estimation. Intelligent Sustainable Systems, 711–729. https://doi.org/10.1007/978-981-19-7663-6_67	Prior model-agnostic explanation of SVR effort estimates
Jayadi, P., Prasetya, D.D., & Widiyaningtyas, T. (2025). Explanation of a Simple Machine Learning Model for Software Effort Estimation using SHAP (SHapley Additive exPlanations). Proc. 9th ICEEIE, 1–5. https://doi.org/10.1109/ICEEIE66203.2025.11254827	Prior SHAP application in software-effort estimation
Stack Overflow Developer Survey (2024). stackoverflow.com/survey	AI tool adoption statistics
GitHub Octoverse (2024). github.com/octoverse	Copilot code contribution data